SELIA · I JORNADA SOBRE SOFTWARE LIBRE E IA ABIERTA · 6 JUL 2026



Inteligencia artificial: programar, documentar y no acabar en un berenjenal

Decisiones trazables (ADRs), IA agnóstica y software libre — con **mod_exelearning** como caso real.

Ernesto Serrano Collado
DevOps en eXeLearning · *Rompiendo cosas desde 1983*

ernesto.es · github.com/erseco

CC BY-SA 4.0

📖 Programar, documentar y no acabar en un berenjenal

La IA ya programa. El reto es no acabar en un berenjenal.

Charla en la **I Jornada sobre Software Libre e Inteligencia Artificial Abierta (seLIA)**.

Ernesto Serrano — *Rompiendo cosas desde 1983*.

- Desarrollador principal del **simulador de Stevie Wonder** para Android.
- **DevOps Engineer** en eXeLearning.
- Coordinador del **Área de Tecnología Educativa**, de la Dirección General de Ordenación de las Enseñanzas, Inclusión e Innovación, en la Consejería de Educación, Formación Profesional, Actividad Física y Deportes del Gobierno de Canarias.

La idea central

La IA te lleva a producción a toda velocidad. El problema no es llegar rápido: es **acordarte de por qué tomaste cada curva**.

El código queda; el **"por qué"** se evapora. Ahí empieza el berenjenal.

La salida: **documentar las decisiones** (no solo el código), y hacerlo **trazable, agnóstico y libre**. Lo cuento con un proyecto real: **mod_exelearning**.

Qué facilita de verdad

Sin hype: hoy la IA ayuda de verdad en tareas concretas.

- **Generar código**: borradores, funciones repetitivas, *boilerplate*.
- **Detectar errores**: explicar trazas, sugerir causas.
- **Pruebas**: tests para código sin cobertura.
- **Documentación**: a partir del código real.

Pero **propone, no decide**. Y va tan rápido que es fácil **perder el rastro**.

El berenjenal

? Velocidad sin rastro = caos

En unas semanas, el proyecto que iba rapidísimo se convierte en un berenjenal:

- El **código** está; el **"por qué"** no.
- Nadie recuerda qué alternativas se descartaron ni con qué evidencia.
- Y en **software libre**, encima: *¿de dónde viene ese código? ¿la licencia y la procedencia están limpias?*

Programar rápido es fácil. **No acabar en un berenjenal** es el trabajo de verdad.

ADRs: la IA apoya la decisión

Un ADR captura el porqué

Un **ADR** (Architecture Decision Record) documenta **una decisión**: no el código, el **porqué**.

Cada uno es un `DEC-NNNN` con secciones fijas: *Contexto · Problema · Opciones · Evidencia · Decisión · Consecuencias · Riesgos · Validación*.

En **mod_exelearning** (plugin de Moodle, **GPLv3**): **60 ADRs** — 53 aceptadas, 4 propuestas, 3 superseded. En el código, cada función **cita su decisión**: `// (DEC-0008)`.

El frontmatter deja rastro

```
id: DEC-0043
titulo: "Detección de GeoGebra calificable"
estado: Aceptada        # Propuesta | Aceptada | Rechazada | Superseded
fecha: 2026-06-22
agentes: [erseco, codex]
herramienta_ia:
  interfaz: codex
  modelo: gpt-5
fuentes: [REPO-071, EXP-014]
```

- El ADR **declara quién decidió** (persona o agente) y **con qué IA y modelo**.
- **"Sin fuente no hay afirmación":** cada dato cita `repo + ruta + commit`, doc oficial o experimento reproducible.
- Ver el ADR completo en el repo: DEC-0043 ↗

Propone la IA, dispone el humano

Los papeles quedan claros

El ADR reparte los papeles:

- La **IA redacta** las *Opciones* y la *Evidencia* (con fuentes).
- El **humano decide**: fija el `estado` (Propuesta → **Aceptada / Rechazada**).

Ejemplo real: **auditorías multi-agente**. En **DEC-0016**, decenas de agentes revisan en 12 dimensiones → **21 hallazgos**; el humano **corrige 18 y difiere 3** con justificación.

La IA **propone**; el humano **dispone**. Y siempre: **revisa el diff**.

Un sistema de trazabilidad completo

Los ADRs no van solos: viven en `research/`, un sistema que hace **todo trazable**.

- `decisiones/` — los ADRs (`DEC-NNNN`).
- `fuentes/` — fuentes citables (repos de referencia, docs): `REPO-NNN` / `FTE-NNN`.
- `experimentos/` — reproducibles (`EXP-NNN`): comando, commit, entorno, métricas.
- `analisis/` , `tareas/` , `status.yaml` , **diario** — trabajo y estado, **append-only**.
- `plantillas/` , `schemas/` , `tools/` — plantilla de ADR, validación e **índices auto-generados**.
- `AGENTS.md` — las reglas del juego para personas y IA.

Cada ADR **cita** sus fuentes y experimentos por su id. Tiras del hilo y llegas a la evidencia.

¿ADR o SDD (spec-kit / openspec)?

❓ Qué vs por qué

No compiten: SDD dice el *qué*; el ADR, el *por qué*.

- **SDD** (Spec Kit, OpenSpec): spec → genera plan, tareas y código. **Caduca** al implementarla.
- **ADR**: registra la decisión y su evidencia. **Append-only**, perdura.

A favor del ADR: agnóstico (markdown+git, **sin lock-in**), auditable, ideal en **código vivo**.
En contra: más manual, no genera código; para una *feature* nueva de cero, SDD estructura mejor.

No es *o uno o otro*: se **combinan** (OpenSpec trae `spec-driven-with-adr`).

Agnóstica, transparente y libre

IA agnóstica: sin atarte a ninguna

El frontmatter registra `interfaz` y `modelo`, así el proceso **no se ata a ninguna IA**:

- **DEC-0043** se decidió con `codex` + `gpt-5`.
- **DEC-0044** (auditoría de bugs) con `claude-code` + `claude-fable-5`.
- Otros, con `claude-opus` … **da igual el motor**.
- Y el mismo proceso vale con **modelos de pesos abiertos** (Llama, Mistral, Qwen, DeepSeek) e incluso de **licencia libre / OSI** (OLMo, IBM Granite), corriendo **en local** (Ollama).

La **decisión** y su **evidencia** sobreviven a la IA que las escribió. Cambias de modelo (o de proveedor) sin perder el hilo: **cero lock-in**.

Software libre = transparencia + contexto

El repo libre es lo que lo hace posible

Que el repo sea **libre y público** (mod_exelearning, **GPLv3**) no es un detalle: es lo que hace todo esto posible.

- **Contexto**: cualquier IA (o persona) lee el repo y **retoma el hilo** — issues, PRs, ADRs, evidencia.
- **Transparencia**: todo es **append-only** y citable; se puede auditar.
- **Coherencia libre**: *pesos abiertos ≠ software libre*; prefiere modelos **abiertos/locales**, no subas datos sensibles y **devuelve al procomún**.

Cierre

Takeaways

1. La IA **programa y documenta**; **tú decides**.
2. Documenta la **decisión** (ADRs), no solo el código.
3. Registra **qué IA y modelo → agnóstico**, sin lock-in.
4. **Repo libre = transparencia + contexto** para cualquier IA.
5. **Sin fuente no hay afirmación**, y **revisa el diff**.

Moraleja

La IA no te saca del berenjenal. **Un buen ADR, sí.**

Programar rápido está al alcance de cualquiera. **Documentar la decisión** —de forma trazable, agnóstica y libre— es lo que te deja dormir tranquilo. Y si el repo es libre, el berenjenal lo desenredamos entre todos.

Enlaces y gracias

Gracias. ¿Preguntas? (y pásate por el stand)

- mod_exelearning — https://github.com/exelearning/mod_exelearning